# Zone: Extending a bin2bin Metamorphic Engine with Code–Data Identification and LLM Assistance

Stefano Bistarelli[0000−0001−7411−9678], Jacopo Di Benedetto, and Francesco Santini[0000−0002−3935−4696]

Dipartimento di Matematica e Informatica, Università degli Studi di Perugia,
Via Vanvitelli 1, 06123 Perugia (PG), Italy
stefano.bistarelli@unipg.it
jacopo.dibenedetto@studenti.unipg.it
francesco.santini@unipg.it

**Abstract.** Zone is a binary-to-binary metamorphic engine for compact Windows x86 and x64 payloads. This extended article retains the original design, Position-Independent Instruction method, and evaluation of 102 inputs under fifteen configurations. It adds symbolic and heuristic code–data identification, an alternative LLM classifier, LLM-assisted assembly transformation, and an implementation of assembly wrapping for both architectures. A new corpus of thirteen benign fixtures supplies predeclared output-buffer checks; six further fixtures provide independent code–data annotations. DeepSeek-assisted transformation produced accepted outputs with correct buffers in 167/168 x64 and 138/144 x86 invocations under separately versioned experiments. The classifier matched all eighteen LLM and six symbolic reference tests on the six identification fixtures. A separate wrapping campaign on the 102 original payloads built 243/306 variants, of which 170 passed the stated functional checks, exposing a substantial gap between construction and observed functionality. Completed earlier static scans and ongoing payload-level LLM experiments are reported as separate cohorts. The assessment distinguishes classification, construction, functionality, static verdicts, similarity, diversity, and cost, without extrapolating raw-binary observations to deployed endpoint protection.

**Keywords:** Binary Rewriting · Metamorphic Code · Code–Data Identification · Large Language Models · Functional Preservation

**Draft note.** Results are frozen at 21:20 CET on 13 November 2026. The completed benign benchmark, earlier scan cohort, and current payload campaign have separate denominators. The full-corpus LLM and current scan campaigns are still in progress.

## 1 Introduction

Code transformations complicate the analysis of malicious software because distinct binary representations may implement the same behavior. Metamorphism is one form of transformation-based evasion [12]: the program's instruction sequence is rewritten, whereas polymorphic encoding typically changes an encrypted representation and its

decoding routine. Both approaches challenge byte-pattern recognition, but they have different implications for the representation available during execution [5,20].

Zone provides a modular environment for examining these effects on compiled Windows x86 and x64 payloads. Its *bin2bin* interface accepts binary input without source code or debugging symbols. The transformations considered here are equal code substitution (EQ), NOP insertion, instruction block permutation (IBP), bogus control flow (BCF), and position-independent instruction (PII). Their individual and combined effects are assessed through detection measurements, functional tests, and changes in size and instruction statistics.

*Relationship to the Conference Paper and Extensions.* This article extends our SECRYPT 2026 study [4]. It retains the engine description, PII design, original experimental protocol, detailed results, and comparison with Shikata Ga Nai as the foundation of the extended investigation. Reused figures, tables, listings, and the algorithm carry source citations. Additional sections address the limitations identified by that study through:

- symbolic and heuristic code–data identification, an alternative LLM-based classifier, and shared deterministic data handling;
- LLM-assisted assembly transformation with zero-shot and two-shot conditions, validation, and feedback;
- integration and assessment of assembly wrapping, an existing technique, with separately assessed x64 and x86 implementations;
- additional evaluation on benign fixtures with predeclared functional oracles and code–data annotations, complemented by separately identified payload and static-scan campaigns. Classification, construction, functionality, similarity, diversity, and cost remain distinct measurements.

The extension complements the original experiments. Its methods and results are identified separately so that the published findings remain available for comparison and are not conflated with new measurements.

The scope is restricted to compact, position-independent payloads rather than complete Portable Executable (PE) files. Full-file rewriting requires additional treatment of section boundaries, imports, relocation information, alignment, and entry points. Those format-dependent issues would complicate the interpretation of experiments intended to compare instruction-level transformations. Signatures in non-code regions of PE files are an additional consideration [22].

Operating on largely executable instruction sequences makes it possible to examine a substantial fraction of each input. It also limits the conclusions: these experiments address a payload-level representation, not the complete delivery and execution of a malicious application. In particular, functional preservation and the detectability of a raw binary must be considered separately.

The published baseline comprises 102 Windows payloads, including 38 x64 and 64 x86 inputs, evaluated under fifteen transformation configurations. Its main observation is a trade-off between changes in the binary representation and the proportion of variants

that remain functional. Some configurations received no malicious verdicts in the VirusTotal scans, but such observations are specific to the file format and testing conditions; they do not establish general resistance to deployed endpoint products.

Sections 2 and 3 retain the background and engine design. The original experiments and representation measurements appear in Sections 4 and 5, followed by their limitations and the SGN comparison (Sections 6 and 7). Sections 8, 9, and 10 describe the additional methods. Sections 11 and 12 define and reserve the extended assessment. Section 13 discusses related work and Section 14 concludes.

## 2 Background

The following background distinguishes metamorphic rewriting from polymorphic encoding and introduces the transformation families used in the baseline evaluation.

### 2.1 Metamorphic Engines

A metamorphic engine constructs a different instruction-level representation of an input program intended to preserve its behavior. Early virus generators used this idea to vary their byte sequences between instances [25]. Functional preservation is a design objective and must be checked; it does not follow merely from the fact that a transformation has produced an executable output.

A key difference is the absence of a mandatory decrypt-and-execute stage [29]. An encrypted payload is normally recovered before execution, while a successfully rewritten payload can execute in its transformed representation. This changes what a memory scan may observe, without implying that the transformed program is undetectable.

Instruction substitution, insertion, and reordering can generate many representations from the same input [29,17]. This diversity complicates the use of fixed byte patterns, and places a substantial burden on the rewriting engine to preserve control flow, data references, and processor state. Transformation-based evasion and its limitations are surveyed in [12].

*Comparison with Polymorphism.* Polymorphic encoding usually combines an encoded body with a variable decoding routine [25]. Shikata Ga Nai is a representative example: variation in its decoding stub changes the static byte sequence, but recurring features of that stub can themselves be used for recognition [18]. Once decoding has completed, the original body may be available to memory analysis. Metamorphic rewriting instead changes the body that is executed [11]; this is a distinction between representations, not a guarantee of runtime evasion.

### 2.2 Morphing Techniques

The established transformation families considered in Zone are EQ, NOP insertion, IBP, and BCF. They can be evaluated separately or in selected combinations. Their antecedents are discussed in Section 13.

**Table 1.** Instruction substitutions and corresponding encodings (x86). Reproduced from Table 1 of [4].

| Original Inst. | Morphed Inst. | Orig. Bytes | Morphed Bytes |
|---|---|---|---|
| `XOR eax, eax` | `MOV eax, 0` | `31 C0` | `B8 00 00 00 00` |
| `SUB eax, 1` | `ADD eax, -1` | `83 E8 01` | `83 C0 FF` |
| `ADD eax, 2` | `SUB eax, -2` | `83 C0 02` | `83 E8 FE` |
| `MOV eax, 0` | `PUSH 0; POP eax` | `C7 C0 00..` | `6A 00 58` |

EQ changes the instruction vocabulary; NOP insertion adds operations intended to leave the relevant computation unchanged. IBP modifies the physical layout of instruction blocks, whereas BCF introduces branches and unreachable regions. PII, described in Section 3.1, further separates instruction placement from logical execution order. These descriptions summarize the design reported in [4].

*Equal Code Substitution.* EQ replaces an operation with an alternative representation intended to have the same effect. For example, the zeroing operations `XOR rax, rax` and `MOV rax, 0x0` have different encodings. Such examples illustrate changes in byte representation; preservation of all relevant state remains an implementation concern.

Table 1 collects the substitution examples used in the conference study, including arithmetic complements and stack-based sequences. Instruction substitution and its use with insertion transformations have a long history [7,8,23].

*NOP/Dead Code Insertion.* Insertion transformations introduce operations intended to leave the program's relevant behavior unchanged. The single-byte `NOP` is one example; Table 2 reproduces the other examples considered in the original implementation.

Some apparent no-ops, including arithmetic with zero, can change processor flags. The original implementation therefore used flag-preserving sequences around such operations, as illustrated in Table 3. The resulting repeated patterns are themselves relevant to detection: changes introduced by an engine can create recognizable regularities [3].

*Instruction Block Permutation.* IBP changes where instruction blocks are stored while retaining their intended logical succession. The original implementation organizes the transformation into the following stages:

1. **Partitioning.** The instruction sequence is divided into blocks; the chosen granularity affects both rewriting overhead and the amount of layout variation.
2. **Connecting blocks.** An unconditional branch links each block to its logical successor.
3. **Changing placement.** Blocks are permuted, while the entry block retains its position.
4. **Updating references.** Control-flow targets are adjusted to the resulting layout.

This transformation changes adjacency without necessarily changing most instructions within a block. Consequently, a detector based on a short pattern internal to an unchanged region may behave differently from one that relies on a longer contiguous sequence. The experiment assesses these changes empirically rather than assuming that every signature is disrupted.

**Table 2.** NOP-like operations and their encodings. Reproduced from Table 2 of [4].

| Example | Compiled Opcode |
|---|---|
| `NOP` | `0x90` |
| `SUB rax, 0x0` | `0x48, 0x83, 0xE8, 0x00` |
| `ADD rax, 0x0` | `0x48, 0x83, 0xC0, 0x00` |
| `MOV rax, rax` | `0x48, 0x89, 0xC0` |
| `LEA rax, [rax + 0]` | `0x48, 0x8D, 0x40, 0x00` |
| `PUSH rax; POP rax` | `0x50, 0x58` |
| `OR rax, rax` | `0x48, 0x09, 0xC0` |
| `XOR rax, 0x0` | `0x48, 0x83, 0xF0, 0x00` |
| `AND rax, rax` | `0x48, 0x21, 0xC0` |
| `INC rax; DEC rax` | `0x48, 0xFF, 0xC0; 0x48, 0xFF, 0xC8` |
| `DEC rax; INC rax` | `0x48, 0xFF, 0xC8; 0x48, 0xFF, 0xC0` |
| `SAR rax, 0x0` | `0x48, 0xC1, 0xF8, 0x00` |
| `SHR rax, 0x0` | `0x48, 0xC1, 0xE8, 0x00` |
| `SHL rax, 0x0` | `0x48, 0xC1, 0xE0, 0x00` |
| `RCL rax, 0x0` | `0x48, 0xC1, 0xD0, 0x00` |
| `RCR rax, 0x0` | `0x48, 0xC1, 0xD8, 0x00` |

**Table 3.** Flag-preserving insertion example. Reproduced from Table 3 of [4].

| Instruction | Compiled Opcode |
|---|---|
| `PUSHF` | `0x9C` |
| `ADD rax, 0x0` | `0x48, 0x83, 0xC0, 0x00` |
| `POPF` | `0x9D` |

*Bogus Control Flow.* BCF adds branches around code that is not intended to execute. The implementation uses conditions with predetermined outcomes to exclude the inserted region from the execution path [10,15]. Table 4 reproduces the illustrative block from the conference article: the condition transfers control past the padding at addresses `0x11`–`0x17`.

1. **Inserted regions.** The engine creates the additional blocks, which may contain instruction sequences or raw bytes.
2. **Predetermined conditions.** Conditions are constructed to select a known branch outcome.
3. **Integration.** The conditions and corresponding branches are added to the instruction stream. Table 4 illustrates the resulting organization.

## 3 Zone Architecture

The name Zone refers to Tarkovsky's *Stalker* and its alternative paths toward a common destination. The open-source implementation[1] was originally explored for both complete PE files and raw payloads. The extended manuscript retains the payload-level scope of

[1] Project repository: https://github.com/jakydibe/Zone.

**Table 4.** Illustrative BCF block. Reproduced from Table 4 of [4].

| Address | Instruction |
|---|---|
| 0x0 | `mov rax, 0x1` |
| 0x7 | `cmp rax, 0x1` |
| 0xb | `je 0x18` |
| 0x11 | `nop` |
| 0x12 | `nop` |
| 0x13 | `nop` |
| 0x14 | `nop` |
| 0x15 | `nop` |
| 0x16 | `nop` |
| 0x17 | `nop` |
| 0x18 | `original code.......` |

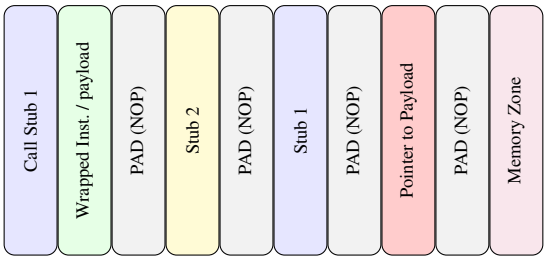



**Fig. 1.** PII payload layout. Adapted from Fig. 1 of [4].

the conference paper [4]; the difficulties of maintaining PE metadata are not evaluated as a separate contribution.

The implementation is written in Python. Capstone disassembles the input, the selected transformations modify the instruction representation, and Keystone assembles the output. The *bin2bin* interface requires neither source code nor Program Database (PDB) symbol files.

### 3.1 Position-Independent Instruction

PII was introduced in the conference paper [4]. It decouples the physical position of an instruction from its place in the intended execution sequence.

*Organization.* Consider an instruction such as `ADD rax, 1`. In the PII representation it is placed in a wrapper rather than relying on adjacency to its successor. The wrapper executes the original operation and transfers control to a redirector. An offset table supplies the location of the next logical instruction. Figure 1 reproduces the overall layout, while the listings below document the components of the original design.

The representation consists of four components:

1. **Base-Address Stub (Stub1).** This component determines the runtime base address and records it in a designated memory location.

2. **Redirector Stub (Stub2).** This component obtains the next logical instruction location from the offset table and dispatches execution to it.
3. **Offset Table.** This contiguous table contains the instruction offsets used by the redirector.
4. **Wrapped Instructions.** Each input instruction is placed in a wrapper intended to maintain the required register, flag, and stack state.

*Execution Sequence.* The conference paper describes execution in three phases:

– **Initialization.** A call enters Stub1 (Listing 1). The return address supplied by that call is used to establish the payload's runtime location.
– **Original operation.** The wrapper in Listing 2 restores the saved state, adjusts the stack, executes the input instruction, and supplies its logical index to Stub2.
– **Dispatch.** The redirector in Listing 3 saves the state it uses, obtains the successor's location from the offset table and base address, and transfers control to that wrapper after restoring the relevant state.

*Physical Placement.* The original implementation permutes complete wrappers, treating each five-instruction wrapper as a unit. The offset table is updated for the resulting placement. Execution still starts with the initial call to Stub1; logical succession is represented by the table rather than by the wrappers' physical adjacency.

**Listing 1.** Base-address component. Reproduced from Listing 1 of [4].

```
pop <random_unused_reg>;    popping 2nd instruction address
mov [<random_unused_reg> - 5 + <init_address>],  <random_unused_reg>; move it to a memory
     slot
push <random_unused_reg>; push just to avoid altering the stack
jmp <random_unused_reg>; return to the caller (2nd instruction)
```

**Listing 2.** Instruction wrapper. Reproduced from Listing 2 of [4].

```
pop rax;                    ; popping rax because we just used it to save jmp address
lea rsp, [rsp + 8];         ; adjusting stack to avoid stack overflow
ORIGINAL_INSTRUCTION;       ; original instruction
push <0x3e8 + counter>;     ; pushing instruction original index + a constant
jmp <stub2_address>;        ; Jumping to 'redirector' stub
```

**Listing 3.** x64 redirector. Reproduced from Listing 3 of [4].

```
push rax;                                       saving original 'rax' register state
push rcx;                                       saving original 'rcx' register state
mov rcx, [rsp + 0x10] ;                         getting the instruction index from the stack.
pushf;                                          saving original CPU flags
sub rcx, 0x3e8; ;                               subtracting a constant to get instr index
inc rcx;                                        increase instruction number
mov rax, [rip + init_addr_offset];              get payload base address
mov rcx, [rax + mem_address - 5 + rcx * 8];     get next instruction address
sub rcx, 0x5;                                   just because base address is 2nd instruction
add rax, rcx;                                   add payload base address to the offset
popf;                                           restore cpu flags
pop rcx;                                        restore rcx value
jmp rax;                                        jmp to next instruction
```

*Architectural Variants.* Listings 3 and 4 show the published x64 and x86 redirectors. In addition to the different register widths, they differ in how the required address is obtained: the x86 version uses the call-based mechanism documented in the original listing rather than x64 RIP-relative addressing.

**Listing 4.** x86 redirector. Reproduced from Listing 4 of [4].

```
push eax;
push ecx;
mov ecx, [esp + 0x8]
pushf;
sub ecx, 0x3e8;
inc ecx;

call <stub2_address + 0x14>;    calling pop eax, 0x14 is pop eax offset from stub2 address
pop eax;                        popping eax puts in eax the value of the instruction pointer

mov eax, [eax + <init_addr_offset>];
mov ecx, [eax + <mem_address> - 5 + ecx * 4];
sub ecx, 0x5
add eax, ecx
popf;
pop ecx;
jmp eax;
```

### 3.2 How Instructions Are Patched

Moving instructions or inserting bytes can invalidate references to their previous locations. The patching phase updates control-flow destinations and other affected references to the modified layout. Branches and calls are therefore part of the rewriting process, not merely instructions that can be copied without regard to their new positions.

This requirement also explains the scope restriction. Rewriting raw instruction sequences already requires consistent target updates; complete PE files introduce additional metadata and layout dependencies. Algorithm 1 reproduces the high-level workflow from [4].

### 3.3 Transformation Pipeline

The original workflow in Algorithm 1 can be read as six stages. The additional analysis and transformation paths are described in Sections 8–10.

1. **Input representation.** Capstone produces instruction objects held in a linked sequence, with an address-indexed dictionary for lookup (Line 4).
2. **Embedded data.** The implementation enables Capstone's `Skipdata` option (Line 5). This does not solve the general problem of identifying inline data, particularly when data bytes also admit an instruction interpretation [10].
3. **Reference tracking.** A label table associates each tracked control-flow operation with its target instruction object. This association is retained as the object's physical address changes.
4. **Transformation and layout update.** The chosen transformations are applied (Line 19), followed by recomputation of instruction addresses (Line 12).
5. **Reassembly of tracked branches.** Keystone assembles the tracked control-flow operations using their updated targets (Line 25).
6. **Output.** The resulting instruction bytes are written as a new raw binary (Line 36).

**Algorithm 1:** High-level Zone workflow. Reproduced from Algorithm 1 of [4].

```
Data: raw_bytes
Result: Transformed binary
global instruction_list ← [];
global instruction_dict ← {};
global label_table ← [];
Function disass
    CAPSTONE.SKIPDATA ← TRUE;
    for each instruction in disasm(raw_bytes)
        instruction_list.append(instruction);
        instruction_dict[instruction.address] ← instruction;
        if instruction is X86.Control_flow_instruction
            jump_address ← extract_address(instruction.op_string);
            label_table.append((instruction, instruction_dict[jump_address]));
Function update_instructions
    new_instruction_dict ← {};
    for each instruction in instruction_list
        address ← instruction.previous_instruction.address + len(instruction.previous_instruction.bytes);
        instruction.address ← address;
        new_instruction_dict[instruction.address] ← instruction;
    instruction_dict ← new_instruction_dict;
Function morph
    equal_code_substitution();
    nop_insertion();
    instruction_block_shuffling();
    bogus_control_flow();
    position_independent_instruction();
Function adjust_label_table
    update_instructions();
    for each entry in label_table
        jmp_addr ← extract_address(entry[0].op_string);
        jmp_addr_new ← entry[1].address;
        new_instruction ← keystone.assemble( “entry[0].mnemonic jmp_addr_new”);
        entry[0] ← new_instruction;
Function main
    disass();
    morph();
    adjust_label_table();
    write_to_file();
```

## 4 Published Experimental Baseline

**Draft note.** The detailed historical plots and their reported numerical values are retained from the preparation manuscript. Some category/value associations in the detection and working-rate plots, and the provenance of the similarity metric, require reconciliation against the conference version and archived records. These are baseline editorial checks, not replacement experiments. See `BASELINE_CHECKS.md`.

This section restates the protocol and results of the conference paper [4]. It introduces no additional measurements. VirusTotal scans supplied verdicts from more than 70 engines, subject to the file-format limitations discussed below.

The corpus contains 102 Windows payloads: 38 x64 inputs and 64 x86 inputs. Most originate from the Metasploit Framework and include reverse and bind shells. This pro-

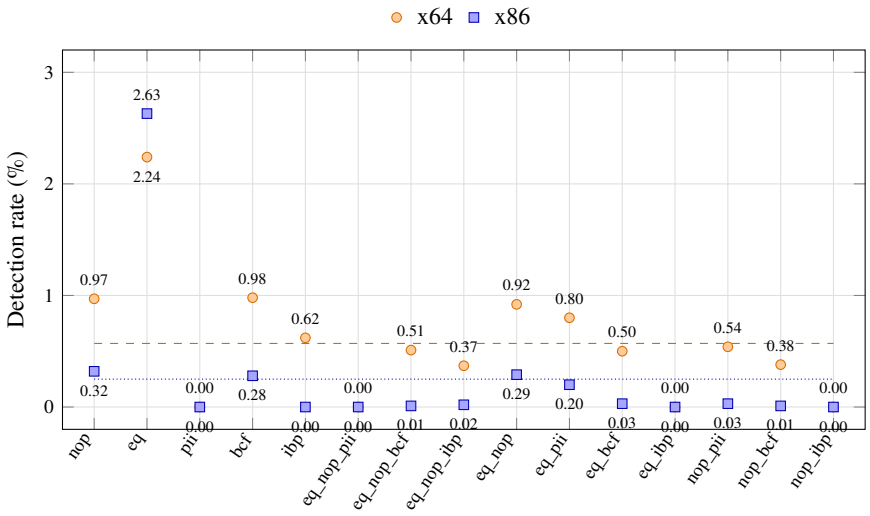



**Fig. 2.** Malicious-verdict fractions in raw-file scans. Dashed reference lines show the reported architecture averages (x64: 0.57%; x86: 0.25%). Data from Fig. 2 of [4]; redrawn for the present format.

vides an identifiable collection of payload types, but does not constitute a representative sample of all Windows malware.

Two scope restrictions are relevant to interpretation:

- **Architecture coverage.** The x64 and x86 sets differ in size and membership. Architecture-level comparisons therefore also reflect differences between the two input collections.
- **Excluded inputs.** Large, DLL-based command-and-control components are outside the payload-level scope. Their PE structure and embedded data introduce additional rewriting dependencies.

*Testing Method.* Each of the fifteen configurations was applied to every input, with ten generated variants per input and configuration to expose variation in the transformation process. The five individual transformations and ten combinations were:

- **Single:** NOP (`nop`), Equal Code Substitution (`eq`), Bogus Control Flow (`bcf`), Instruction Block Permutation (`ibp`), and Position-Independent Instruction (`pii`).
- **Combined:** `eq_nop_bcf`, `eq_nop_ibp`, `eq_nop_pii`, `eq_nop`, `eq_bcf`, `eq_ibp`, `eq_pii`, `nop_bcf`, `nop_ibp` and `nop_pii`.

The *detection rate* for an individual file is the fraction of scanning engines that returned a malicious verdict; summaries aggregate these per-file measurements. Figure 2 presents the published transformed-file results. The corresponding baseline averages for the original files were 21.67% for x64 and 22.63% for x86.

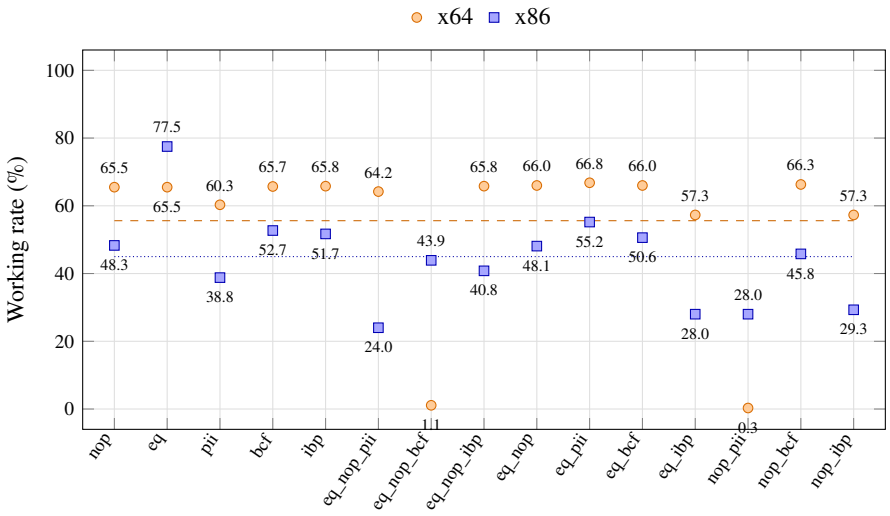



**Fig. 3.** Variants passing the published functional checks. Dashed reference lines show the reported architecture averages (x64: 55.6%; x86: 45.0%). Data from Fig. 3 of [4]; redrawn for the present format.

The plotted observations vary by transformation and architecture. Several configurations involving PII received no malicious verdicts in these raw-file scans, whereas EQ retained more detections. These measurements describe the tested representations and submission conditions; they should be interpreted together with the functional results.

The *working rate* is the proportion of variants that passed the original execution checks (Figure 3). Those checks observed the expected connection for reverse shells, port opening for bind shells, and exception-free termination for the remaining generic cases. The tests are empirical functional checks rather than proofs of semantic equivalence. The conference paper reported greater overall stability for EQ and lower stability for PII, particularly in the x86 collection; the results also varied substantially across combined configurations.

*Evaluation Limitations.* A raw `.bin` submission is not a complete executable application. Engines may apply a different analysis path to such input, omit execution-based analysis, or decline to analyze an unsupported representation. Low detection fractions therefore cannot be interpreted as an evaluation of complete antivirus or endpoint detection and response (EDR) products during deployment.

Submitting original and transformed payloads under the same format assumptions provides a limited basis for comparing the representations in this corpus. It does not remove format-dependent biases or establish behavior under other execution conditions. The published results should be read within those boundaries [4].

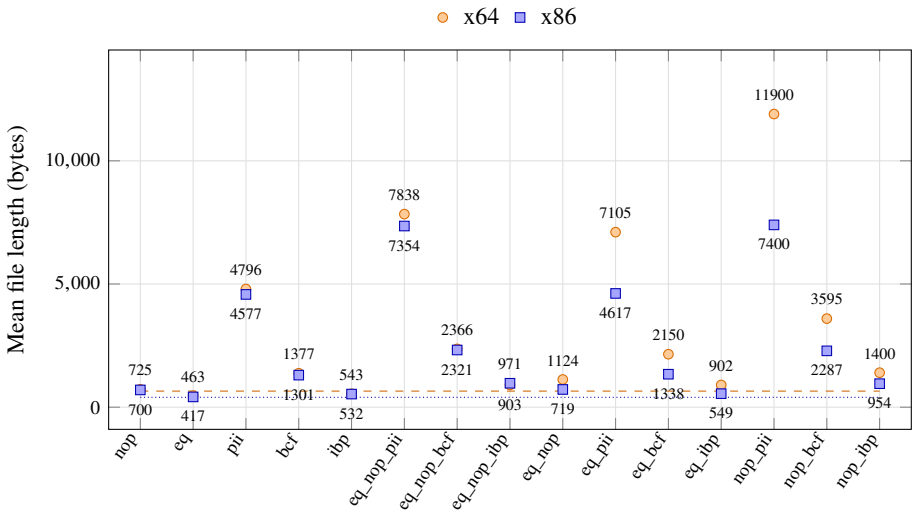



**Fig. 4.** Mean transformed-file length. Reference lines indicate the original-file means (x64: 652.632 bytes; x86: 400.750 bytes). Data from Fig. 4 of [4]; redrawn for the present format.

## 5 Size and Statistical Measurements

The conference paper also characterized how the transformed representations differ from their inputs [4]. Its three additional measurements were:

- Payload length variation.
- Delta entropy.
- Cosine similarity.

*Payload Length Variation.* Let $L_i$ denote the byte length of a transformed file and let $n$ be the number of files considered for a configuration. The reported mean length is:

$$\overline{L} = \frac{1}{n} \sum_{i=1}^{n} L_i$$

The size results in Figure 4 show substantial growth for PII and BCF configurations. Some EQ-containing configurations instead produced smaller average outputs than the original-file reference. The conference paper associated this observation with the implementation's treatment of data during disassembly; the size measurement alone does not establish that any removed bytes were semantically irrelevant.

*Delta Entropy.* The reported entropy change compares the transformed representation, $H_{tech}$, with its original, $H_{orig}$, using the percentage difference:

$$\Delta H\ (\%) = \frac{H_{tech} - H_{orig}}{H_{orig}} \times 100 \tag{1}$$

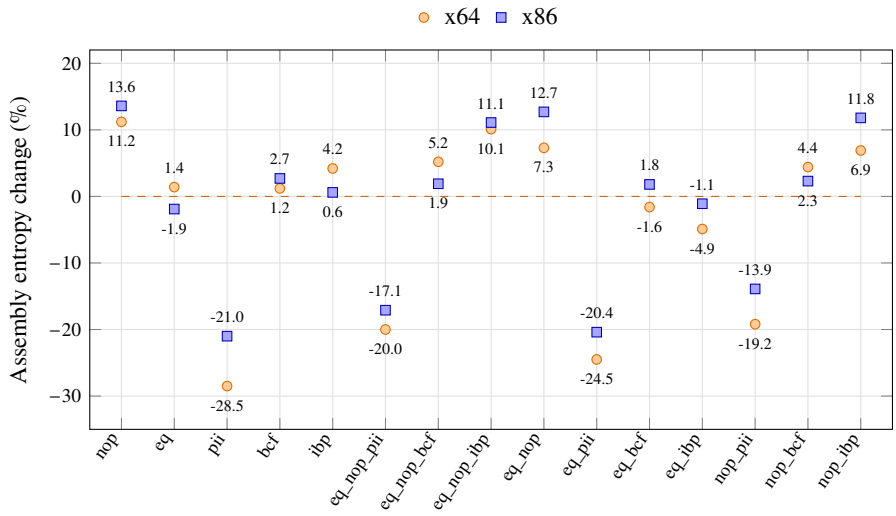



**Fig. 5.** Percentage entropy change for tokenized assembly; the reference level is zero. Data from Fig. 5 of [4]; redrawn for the present format.

The measurements were computed on raw bytes and on tokenized assembly. Figures 5 and 6 retain the published results. The assembly-based representation also permits comparison with the metrics discussed by Mohseni et al. [19].

In the assembly results, NOP insertion increased entropy for both architectures, while PII decreased it. The latter observation is consistent with the repeated bookkeeping instructions in the wrapper representation. A small entropy change does not, by itself, imply a small change in control-flow organization. The conference paper illustrated this limitation using BCF and IBP:

- **BCF.** A small entropy difference coexisted with greater length and a changed opcode distribution (Figure 7).
- **IBP.** Moving blocks can change their layout while retaining much of their instruction-frequency distribution.

Entropy is consequently treated as a descriptive statistic of the chosen representation, not a standalone measure of structural complexity or functional preservation.

*Cosine Similarity.* The frequency vectors $A$ and $B$ represent the opcode distributions of an original file and its transformed variant. Their cosine similarity is:

$$\mathrm{sim}(A,B) = \frac{A \cdot B}{\|A\|\,\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\,\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Here $A_i$ and $B_i$ are the frequencies of opcode $i$, and $n$ is the size of their joint vocabulary. The score summarizes agreement between the two frequency profiles. Its use

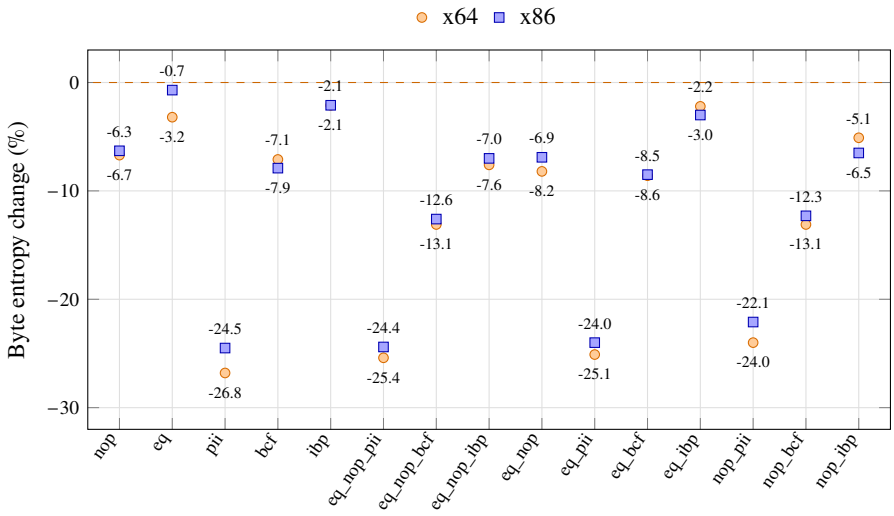



**Fig. 6.** Percentage entropy change for raw bytes; the reference level is zero. Data from Fig. 6 of [4]; redrawn for the present format.

in assembly-obfuscation evaluation is discussed in [19]; it does not establish that the programs have the same behavior.

Instruction-frequency agreement is insufficient to validate execution: a small error can change a program's behavior while leaving that distribution almost unchanged. For this reason, the baseline used the execution checks described in Section 4 to assess functionality separately.

Figure 7 shows higher similarity for EQ and IBP than for the insertion-heavy configurations. This is consistent with their different effects on the instruction vocabulary. The larger size changes in Figure 4 provide complementary information about the additional instructions; neither measurement should be substituted for a functional test.

## 6 Limitations and Defensive Implications

The following limitations concern the original implementation and experiments [4]. Sections 8–10 describe subsequent work motivated by these observations; the residual limits of the added paths are discussed in Section 11.4.

- **Instruction coverage.** Less common instructions, including the example `JRCXZ`, can require handling not exercised by this corpus. Testing a fixed collection of 102 inputs does not establish complete x86/x64 coverage.
- **Code–data ambiguity.** Embedded strings may partly decode as instructions. Table 5 illustrates an input in which the bytes of `.exe` are interpreted as a branch. Rewriting the resulting instruction interpretation can corrupt the embedded string.

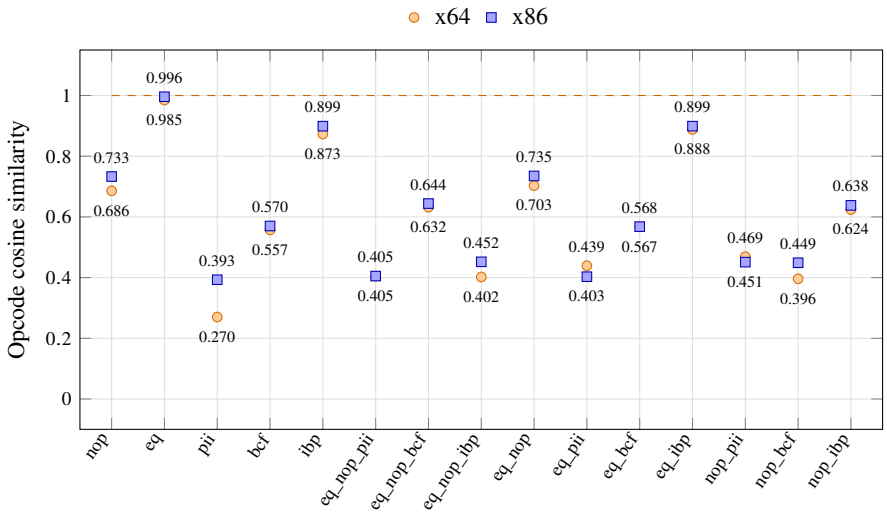



**Fig. 7.** Cosine similarity between original and transformed opcode-frequency vectors; the reference level is one. Data from Fig. 7 of [4]; redrawn for the present format.

**Table 5.** Inline-data example affected by rewriting. Reproduced from Table 5 of [4].

| Bytes | Instruction |
|---|---|
| `0x109 b'\xff\xd5'` | call rbp |
| `0x10b b'c'` | .byte 0x63 |
| `0x10c b'a'` | .byte 0x61 |
| `0x10d b'l'` | insb byte ptr [rdi], dx |
| `0x10e b'c'` | .byte 0x63 |
| `0x10f b'.exe'` | js 0x178 |
| `0x113 b'\x00'` | .byte 0x00 |

Accurate recovery of instructions and references is a broader difficulty in static binary analysis [28]. General limitations of disassembly do not establish that perfect results are impossible on a bounded test corpus; they also do not excuse uncharacterized implementation failures. The reported working rates describe this implementation and these inputs.

The two examples below document the observed code–data distinction. They illustrate one failure mechanism, not a general guarantee about other payloads.

In Table 5, `calc.exe` is stored at the end of the input. Although some characters are represented as data, part of the string receives an instruction interpretation. This explains why the original implementation could modify bytes that were meant to remain data.

Table 6 shows the other published example, in which the string occurs as an immediate operand at address `0x8e`. The reported transformation did not encounter the same inline-data failure. This observation is specific to that example and does not imply general immunity from rewriting errors.

**Table 6.** Immediate-operand example. Reproduced from Table 6 of [4].

| Bytes | Instruction |
|---|---|
| `0x8d b'Q'` | push rcx |
| `0x8e b'H\xb9calc.exe'` | movabs rcx, 0x6578652e636c6163 |
| `0x98 b'Q'` | push rcx |
| `0x99 b'H\x8d\x0c\$'` | lea rcx, [rsp] |
| `0x9d b'H1\xd2'` | xor rdx, rdx |
| `0xa0 b'H\xff\xc2'` | inc rdx |
| `0xa3 b'H\x83\xec('` | sub rsp, 0x28 |
| `0xa7 b'\xff\xd0'` | call rax |

### 6.1 Mitigations and Countermeasures

Changing the binary representation does not remove a program's observable actions. Defensive analysis can consider memory protections, execution patterns, API activity, and network behavior in addition to static bytes. Recognition of a transformation engine is another possibility: repeated artifacts of a common toolchain can persist across outputs, as documented for Shikata Ga Nai [18]. Statistical or learned detectors must also account for benign programs that share similar features. These are defensive considerations arising from the study, not results of an additional detection experiment.

## 7 Comparison with Shikata Ga Nai

The conference paper [4] compared Zone with the Shikata Ga Nai (SGN) polymorphic encoder [13]. SGN combines additive-feedback encoding with a variable decoding stub. The comparison included the original x86 encoder and an unofficial x64 implementation [9]; recognition of the x86 stub has been discussed in [18].

The published aggregate static-scan observations were:

- **x64:** SGN achieved 0%, while Zone averaged 0.57% (reaching 0% with the PII technique).
- **x86:** SGN had a 10.22% detection rate, while Zone achieved 0.25% (also reaching 0% with PII).

The encoders also differ in the representation available after decoding. The conference paper submitted memory dumps obtained during SGN execution to VirusTotal and reported verdicts matching those for the unencoded inputs [4]. This observation is consistent with exposure of the original representation after decoding and with prior work on extracting attack code from memory [11].

Zone's intended execution model retains the rewritten representation. The dump experiment nevertheless measures the recognizability of submitted memory contents; it is not a direct measurement of an endpoint product's runtime detection. Claims about deployed EDR behavior require evidence beyond this comparison.

# 8 Code–Data Identification and Data-Preserving Rewriting

## 8.1 Problem and Representations

For an input byte sequence $B = (b_0, \ldots, b_{n-1})$, identification returns candidate data intervals $[s_j, e_j)$ in payload-relative coordinates, with optional kinds such as ASCII, UTF-16LE, or another data object. Acceptance for rewriting additionally depends on reference evidence and implementation support. The original inline-data and immediate-operand examples in Tables 5 and 6 motivate this added layer.

The implementation distinguishes a model's reported intervals, regions accepted by deterministic checks, and regions actually extracted. These sets need not coincide. An unsupported reference can prevent processing of a correctly identified object, while filtering can make different proposals yield identical final binaries. Evaluation retains the output stage associated with each observation.

## 8.2 Symbolic and Heuristic Identification

The non-LLM path combines CFGFast candidate blocks and inferred data with bounded symbolic exploration. Observed instruction starts are expanded into their decoded byte spans, and newly observed entries can seed a further graph-recovery pass. This prevents interior bytes of an observed instruction from being treated as unvisited data solely because only its start address was recorded.

The initial state leaves uninitialized registers and memory symbolic; it does not reproduce a complete Windows process or all API behavior. Time, state/block-step, pending-state, and solver limits bound the analysis. Reports record termination reasons, unresolved states, and errors. Missing observations therefore do not prove that a region is data or unreachable.

Printable ASCII and UTF-16LE patterns supply additional candidate evidence. Observed code spans prevent their acceptance under the string heuristic. For strings overlapping candidate graph code, additional structural corroboration is required. Graph-inferred data and bytes outside recovered blocks retain different evidential categories; the latter are uncertain rather than a complete data partition. The available analysis limits and candidate categories accompany the classification result.

## 8.3 LLM Identification and Deterministic Handling

The alternative path presents hex bytes, a heuristic linear disassembly, and printable candidates to the model. It requests structured intervals instead of rewritten code. The parser checks bounds, positive lengths, and overlap. Reference scanning and data handling remain deterministic; a proposal is applied to a fresh engine instance so that rejection does not reuse partially modified state.

Accepted data contents and supported references are retained through layout changes. The engine finalizes the supported references once the code layout is known and checks integrity of the preserved data. These checks do not establish that every reference in an arbitrary input has been recovered. Architecture-specific operands

require separate x64 and x86 implementations even though the conceptual contract is shared.

An invalid proposal can trigger feedback and another request in the general pipeline. The reported classifier experiments specify their actual call budgets separately from the transformation experiments. The classifier's default worked-example condition is not labeled zero-shot or two-shot merely because those conditions are compared in assembly transformation.

Caching reuses identified regions for an input identity while references are recomputed for the working representation. Timing must distinguish new analysis from cached results. On curated fixtures, independent annotations allow reference-based assessment; on the selected real payloads, agreement and functional checks remain different forms of evidence. The experiments in Section 12 explicitly retain that distinction.

## 9 LLM-Assisted Assembly Transformation

### 9.1 Task, Representation, and Model Role

Zone delegates one of EQ, NOP insertion, BCF, or IBP to an LLM, requesting a complete modified assembly representation. PII and wrapping remain native transformations. The model generates an instruction proposal; deterministic preparation, assembly, data handling, and validation remain part of the evaluated pipeline.

The engine prepares symbolic labels for relevant code locations and keeps the data representation outside the generated instruction text. Original bytes, the prepared intermediate representation, and final output identity are recorded separately. In particular, similarity to the intermediate input does not measure every change from the original file. The experiments request the model identifier `deepseek-flash`. Version names assigned by a local alias or pricing table are not independent confirmation of the provider's model version.

### 9.2 Zero-Shot, Two-Shot, and Call Budgets

The initial zero-shot request supplies a textual specification without worked examples; the two-shot request adds two input/output examples. Example identities and prompt versions are part of the condition. Repeated run numbers do not imply controlled or independent service seeds. After rejection, the general pipeline can supply the previous answer and validation feedback, so any later attempt is identified as feedback-assisted.

The benign benchmark deliberately disables these internal retries: each invocation allows one generation, and each input/family/shot cell has three invocations in total, including failures. No failed invocation is replaced. The ongoing payload campaign instead uses one target output per cell and a driver supporting up to three generation attempts. These are different experimental designs and are not merged into one success percentage.

### 9.3 Validation and Independent Functional Assessment

Structural checks cover assembly, decoding coverage, tracked control flow, data consistency, and family-specific requirements. A proposal can be rejected because it is malformed, loses required structure, or fails to apply the requested transformation. Structural acceptance alone does not establish behavioral equivalence.

For the new benign benchmark, functional verification follows generation in a separate harness and compares the complete output buffer against its predeclared expected contents. It is not a generation-time selection oracle. This differs from the earlier MessageBox pilot, where the dialog verifier was included in output acceptance. That earlier pilot is retained as a separate cohort.

Records preserve accepted and rejected invocations, transport failures, hashes, timing, usage, and locally estimated cost. Reusing a completed output is distinguished from a new model call. A correct buffer is empirical evidence for the fixture's specified observable, not a proof of all-state equivalence or correctness on unrelated payloads.

## 10 Integration of Assembly Wrapping

### 10.1 Existing Technique and Implementation Scope

Assembly wrapping is an existing technique based on alternative instruction interpretations [1]. The contribution here is its integration into Zone and empirical assessment. The instruction view obtained by static decoding can differ from the stream followed during execution; representation measurements must state which view they use.

The implementation now supports both x64 and x86. The x86 path was added after the first benchmark snapshot and is assessed against a separately identified version. Register width, instruction encodings, and addressing require architecture-specific treatment; an x64 outcome is not transferred to x86 by assumption.

### 10.2 Integration and Assessment Boundary

Wrapping depends on an adequate instruction interpretation, data preparation, and consistent handling of supported references. Some regions may remain outside the main wrapped representation, and construction may be refused on unsupported inputs. Availability, construction success, empirical functionality, size, and diversity are therefore distinct outcomes.

The benign x64 and x86 fixture results and the separate 102-payload campaign appear in Section 12. The latter includes unbuilt and non-working outputs. Functional results on the curated fixtures are not used as evidence of universal payload support. The older selected x64 wrapping/static-scan cohort is also identified separately, including its construction failures and duplicate outputs.

## 11 Additional Experimental Methodology

### 11.1 Corpora, Frozen Versions, and Observables

Three additional cohorts complement the original study. The benign functional corpus contains seven x64 and six x86 fixtures covering arithmetic, branches, loops, local calls,

mixed control flow, strings, and an x64 address-acquisition case. Expected writes are declared before transformation in a 256-byte buffer; all remaining bytes must retain their `0xCC` sentinel. A separate C runner executes the code and returns the complete buffer. All thirteen originals pass their checks before transformation.

Six further classifier fixtures have independent data-range annotations: two multi-region cases (one per architecture), an x64 binary table, two inputs without data, and an x64 case containing ASCII-like code that must not be classified as data. The model receives neither these annotations nor the other method's answer. These fixtures support reference-based assessment; they do not represent the diversity of arbitrary Windows payloads.

Snapshot B1 (13 November, 13:07 CET) is used for the benign native matrix and x64 LLM runs. Snapshot B2 (14:05 CET) is used for the x86 LLM and wrapping tests and the classifier evaluation. The payload campaign uses the separately named B3 copy, whose hashed rewriting and LLM modules match B2. Snapshot identity, architecture, corpus, and call policy remain associated with each result. B1/B2 architecture comparisons consequently do not isolate architecture alone. The current full-payload matrix comprises 38 x64 and 64 x86 inputs with three wrapping runs each; all 102 originals pass the stated current harness checks. Its network, port, dialog, or clean-exit observables are weaker than the benign buffer oracle.

### 11.2 Counting, Classification, and Functional Measures

For $N$ planned cases, $B$ constructed outputs, and $W$ successful functional checks, we report $B/N$, $W/N$, and $W/B$ separately. Unbuilt cases remain in planned-case denominators. Repeated tests of one failed output do not become independent variants. The benign LLM matrix uses three single-generation invocations per cell; rejected answers and transport failures remain in its denominator. The ongoing payload driver has a separate retry policy, with completed cells and generation attempts counted independently.

Identification uses exact equality of annotated and reported interval sets, complemented by missed data bytes and code bytes classified as data. Data-positive precision and recall are $|P \cap G|/|P|$ and $|P \cap G|/|G|$ for predicted and reference byte sets $P, G$. Empty-data cases are assessed through exact empty-set decisions; undefined precision/recall are not assigned favorable numerical values. For real payloads without independent labels, the comparison is agreement and downstream functionality. Reported LLM proposals, symbolically extracted regions, and final binary hashes are different output stages and are not treated as interchangeable annotations.

### 11.3 Similarity, Static Verdicts, and Cost

The additional LLM score is the normalized instruction-sequence matching ratio from `SequenceMatcher`, $2M/(u+v)$ for $M$ matched elements and sequence lengths $u, v$ [21]. NOP-like operations are filtered in that condition, so a value of one does not imply byte identity. Table 8 averages this score over accepted outputs only. Exact diversity counts unique hashes, both within conditions and across the cohort. Neither score establishes semantic equivalence. The historical cosine and entropy measurements remain in Section 5.

For completed static scans, $d_i = m_i/a_i$ is the per-file fraction of malicious verdicts, and $q$ is the fraction of files with any malicious verdict. The completed earlier 2026 cohort below uses all 75 reported engine responses in $a_i$; unsupported formats, timeouts, and failures are separately disclosed. These are not benign verdicts. A pending scan has neither a measured $d_i$ nor a zero detection result. Scan deduplication is per hash, while a shared hash retains membership in every experimental group to which it belongs. Group counts are therefore not summed to obtain the union.

Time and cost are taken from the saved records. Costs are local tariff estimates rather than invoices; failed invocations are included. The classifier payload-arm medians include different preparation and analysis work and are not a controlled claim of general speedup. Evidence tables, source hashes, and experiment settings are frozen at 21:20 CET on 13 November 2026. Later writes to active campaign logs are outside this manuscript snapshot.

### 11.4 Validity of the Extensions

Bounded exploration, missing environment behavior, reference support, and shared deterministic filtering limit attribution to the classifier alone. Buffer equality exercises a specified observable, while a successful connection or dialog supplies narrower evidence. Different snapshots, input selection, model aliases, and scan dates limit causal comparisons. The benign corpus is deliberately small and structured; its outcomes are not extrapolated to the unfinished payload LLM matrix or to deployed endpoint products.

## 12 Additional Experimental Results

The detailed conference results in Sections 4–7 remain the historical foundation. This section adds completed benign experiments, the current full-corpus wrapping and identification campaigns, and a separately identified earlier 2026 scan cohort. The payload-level LLM matrix and its new scan campaign remain partial at the stated cutoff.

### 12.1 Benign Functional Benchmark

Table 7 reports the native matrix. All x64 methods pass 21/21 fixture runs. On x86, EQ, NOP, and BCF pass 18/18; B1 IBP builds 17/18 outputs but none pass the buffer check, with an observed construction exception in the remaining case. The B2 x86 wrapping port passes 18/18. These counts characterize the specified fixture oracle and versions, rather than universal support.

### 12.2 DeepSeek Transformation on the Benign Corpus

The model is invoked on all seven x64 and six x86 fixtures, four families, two shot conditions, and three invocations per cell. The result is 168 x64 invocations under B1 and 144 x86 invocations under B2. Respectively 167 and 138 yield accepted outputs, all

**Table 7.** New native fixture results. *N*: attempts; *B*: builds; *W*: correct complete buffers; *U*: distinct built-output hashes. B1/B2 identify different frozen engine versions.

| Arch. | Family | Version | *N* | *B* | *W* | *U* |
|---|---|---|---|---|---|---|
| x64 | BCF | B1 | 21 | 21 | 21 | 21 |
| x64 | EQ | B1 | 21 | 21 | 21 | 16 |
| x64 | IBP | B1 | 21 | 21 | 21 | 17 |
| x64 | NOP | B1 | 21 | 21 | 21 | 21 |
| x64 | WRAP | B1 | 21 | 21 | 21 | 21 |
| x86 | BCF | B1 | 18 | 18 | 18 | 18 |
| x86 | EQ | B1 | 18 | 18 | 18 | 14 |
| x86 | IBP | B1 | 18 | 17 | 0 | 16 |
| x86 | NOP | B1 | 18 | 18 | 18 | 18 |
| x86 | WRAP | B2 | 18 | 18 | 18 | 18 |

of which pass the independent buffer check. Five invocations are rejected by the family-specific validator and two encounter transport errors. No failed invocation is replaced. The x86 IBP figures include both types of failure; they are not all semantic execution failures.

The x64 outputs contain 108 distinct hashes and cost an estimated $0.5205; x86 contains 99 distinct hashes and costs $0.5132. These union counts differ from sums of per-condition uniqueness because outputs can recur across conditions. EQ in particular has repeated outputs. The NOP similarity of one follows the stated filtering rule and is consistent with byte differences.

On x86 IBP, the observed result changes from 13/18 in the zero-shot condition to 17/18 with two examples; x64 IBP instead records 21/21 and 20/21. The small number of input fixtures and differing versions do not support a general claim that two-shot prompting is superior. The controlled benchmark supports bounded functional observations and exposes construction, diversity, and cost differences.

### 12.3 Reference-Based Code–Data Identification

On the six classifier fixtures, all eighteen LLM invocations and all six symbolic runs exactly match the independent interval annotations (Table 9). Both methods record zero missed data bytes and zero code bytes classified as data. The three data-positive fixtures have exact precision/recall on these cases; the three empty-data cases are evaluated as correct empty sets. All resulting executions pass their buffer checks. These are six curated inputs, not twenty-four independent program samples.

An earlier three-case benign data-handling check also records 9/9 exact LLM results. It remains separate from the six-fixture comparison. For the six-fixture cohort the total model cost is approximately $0.0059, excluding the earlier cohort and pilot calls.

### 12.4 Wrapping on the Original 102 Payloads

The separate B3 campaign attempts three prepared wrapping variants for every original input: 114 x64 and 192 x86 attempts. All 102 unmodified inputs pass the current

**Table 8.** Benign DeepSeek transformation. $W/N$ includes rejected and failed invocations. $S$ is mean sequence similarity over accepted outputs; $U/B$ is unique hashes per built output within the condition. Time is mean invocation seconds; cost is the sum of local USD estimates.

| Arch. | Family | Shots | $W/N$ | $S$ | $U/B$ | Time | Cost |
|---|---|---|---|---|---|---|---|
| x64 | BCF | 0 | 21/21 | 0.574 | 18/21 | 43.0 | 0.107 |
| x64 | BCF | 2 | 21/21 | 0.575 | 20/21 | 35.6 | 0.088 |
| x64 | EQ | 0 | 21/21 | 0.831 | 12/21 | 28.8 | 0.064 |
| x64 | EQ | 2 | 21/21 | 0.832 | 11/21 | 29.4 | 0.068 |
| x64 | IBP | 0 | 21/21 | 0.783 | 14/21 | 25.5 | 0.062 |
| x64 | IBP | 2 | 20/21 | 0.779 | 15/20 | 30.8 | 0.077 |
| x64 | NOP | 0 | 21/21 | 1.000 | 18/21 | 11.9 | 0.027 |
| x64 | NOP | 2 | 21/21 | 1.000 | 20/21 | 11.7 | 0.028 |
| x86 | BCF | 0 | 18/18 | 0.573 | 18/18 | 47.9 | 0.103 |
| x86 | BCF | 2 | 18/18 | 0.580 | 17/18 | 43.8 | 0.095 |
| x86 | EQ | 0 | 18/18 | 0.852 | 11/18 | 29.0 | 0.056 |
| x86 | EQ | 2 | 18/18 | 0.843 | 11/18 | 33.3 | 0.067 |
| x86 | IBP | 0 | 13/18 | 0.705 | 9/13 | 31.0 | 0.052 |
| x86 | IBP | 2 | 17/18 | 0.739 | 12/17 | 32.5 | 0.071 |
| x86 | NOP | 0 | 18/18 | 1.000 | 18/18 | 15.1 | 0.032 |
| x86 | NOP | 2 | 18/18 | 1.000 | 18/18 | 18.6 | 0.039 |

**Table 9.** Independent classifier fixtures. $D$ is annotated data bytes. Exact interval sets are compared against predeclared references. Mean times are per run in seconds.

| Arch. | Case | $D$ | LLM exact | Symbolic exact | LLM time | Sym. time |
|---|---|---|---|---|---|---|
| x64 | databait | 0 | 3/3 | 1/1 | 4.6 | 2.4 |
| x64 | multidata | 11 | 3/3 | 1/1 | 4.2 | 2.6 |
| x64 | nodata | 0 | 3/3 | 1/1 | 2.1 | 2.4 |
| x64 | table | 16 | 3/3 | 1/1 | 2.6 | 2.4 |
| x86 | multidata | 11 | 3/3 | 1/1 | 4.4 | 2.5 |
| x86 | nodata | 0 | 3/3 | 1/1 | 1.9 | 2.3 |

class-specific harness checks. Table 10 includes every planned wrapping case. Overall, 243/306 outputs are constructed and 170/306 pass the harness. Among constructed outputs, 170/243 pass. The aggregate is descriptive; the architecture-specific counts retain the relevant failure stages.

All 63 constructed x64 outputs pass, while 51 attempts do not build. On x86, 180 outputs build and 107 pass; the 73 initial failures also fail in the saved retest records. Retests are not additional variants. This contrasts with the complete success of wrapping on the much smaller benign fixture corpus and demonstrates why build rate and conditional working rate are reported separately. The earlier selected x64 cohort, with 200/260 builds and 198/200 passing outputs, is not substituted for this full-corpus campaign.

**Table 10.** Full original-corpus wrapping campaign, B3. The harness observables are distinct from the benign exact-buffer oracle. *U* counts distinct constructed hashes.

| Arch. | Inputs | $B/N$ | $W/N$ | $W/B$ | $U$ |
|---|---|---|---|---|---|
| x64 | 38 | 63/114 | 63/114 | 63/63 | 63 |
| x86 | 64 | 180/192 | 107/192 | 107/180 | 144 |

### 12.5 Identification on Nineteen Selected Payloads

The payload identification comparison selects thirteen x64 and six x86 inputs for which the symbolic preparation reports extracted data. Both arms build 19/19 outputs and pass 17/19 harness checks. Symbolic extracted ranges and reported LLM proposals agree exactly in 9/19 cases, whereas the final binary hashes agree in 16/19. The stages being compared are different; neither count is reference accuracy on these unlabeled payloads. Their divergence also shows why final byte identity cannot be used to infer identical classification.

The recorded median wall times are 123.95 seconds for symbolic preparation and 59.88 seconds for the LLM arm; the latter costs an estimated $0.2218 across the nineteen recorded runs. Different preparation, caching, and analysis budgets limit performance comparison. The common failures and agreement concern this selected set and the current harness, not the whole identification population.

### 12.6 Completed Earlier Static-Scan Cohort

The earlier 2026 manifest contains 239 file entries and 220 distinct hashes, with completed results for all 220. Twelve distinct hashes receive at least one malicious verdict. Table 11 joins results to each group's hash membership, retaining shared hashes in every applicable row. Therefore the row counts need not sum to 220. This avoids assigning a shared output solely to the first group in which it appeared.

The 197 unique wrapping outputs in this selected x64 cohort receive no malicious verdicts. Thirteen contemporaneous original controls all receive malicious verdicts, with mean 26.62/75 engines. This describes constructed raw files from that cohort, not the availability or functionality of all wrapping attempts. Across its 16,500 reported mutant-engine responses, 3,259 are format-unsupported, 283 time out, and 73 fail. Zero malicious verdicts must be interpreted with those unresolved responses and the raw-file scope visible.

The earlier MessageBox LLM pilot has 24 final passing outputs but includes generation-time dialog selection and a restarted failed run; its recorded accepted artifacts do not provide the same denominator as the single-attempt benign benchmark. The differing scan outcomes across its small groups likewise do not establish a general prompting ranking. No benign fixture is submitted to the scan service in the new functional benchmark.

An audit of the historical archive finds that 6,972 of the 15,021 mutant records no longer have a locally available artifact for hash verification; the 102 original records correspond to 90 distinct hashes. The conference results remain published observations,

**Table 11.** Completed earlier 2026 raw-file cohort. $q$ is detected hashes over completed hashes; $\overline{d}$ is the percentage of all reported engine responses that are malicious, averaged per file. $\overline{m}$ is mean malicious-engine count. These are separate measures.

| Group | Hashes | $q$ | $\overline{d}$ (%) | $\overline{m}$ |
|---|---|---|---|---|
| LLM data only | 4 | 4/4 | 23.33 | 17.50 |
| LLM data + EQ | 1 | 0/1 | 0.00 | 0.00 |
| LLM BCF / 0 | 3 | 1/3 | 1.33 | 1.00 |
| LLM BCF / 2 | 3 | 3/3 | 6.67 | 5.00 |
| LLM EQ / 0 | 2 | 0/2 | 0.00 | 0.00 |
| LLM EQ / 2 | 2 | 0/2 | 0.00 | 0.00 |
| LLM IBP / 0 | 2 | 1/2 | 3.33 | 2.50 |
| LLM IBP / 2 | 3 | 3/3 | 3.11 | 2.33 |
| LLM NOP / 0 | 3 | 0/3 | 0.00 | 0.00 |
| LLM NOP / 2 | 3 | 1/3 | 2.22 | 1.67 |
| Wrapping (prior x64) | 97 | 0/97 | 0.00 | 0.00 |
| Wrapping combined (prior x64) | 100 | 0/100 | 0.00 | 0.00 |

while these losses and duplicates limit retrospective verification and independence. They are not treated as newly repeated experiments.

### 12.7 Ongoing DeepSeek Campaign: Snapshot Only

At 21:20 CET, the new payload transformation ledger contains 30 finished run records covering 30 cells of a planned 816 (102 inputs, four families, two shot conditions): eleven accepted builds and nineteen failed runs. Its records account for 35 API calls where available and an estimated $0.7538. Transport failures can leave incomplete usage information. Observed rejection categories include truncated responses and structural validation failures. This is progress accounting, not a final construction or functional rate for the planned matrix; a recorded build is not a passed execution test.

The associated current scan ledger records 208 uploaded hashes and no completed verdict records at the cutoff. These submissions are kept separate from Table 11. Pending current scans supply no measured detection rate. The frozen evidence and source hashes accompany this manuscript so that later campaign writes do not silently change the reported snapshot.

## 13 Related Work

Instruction-level obfuscation has been studied in both offensive and defensive settings [26,6]. Earlier engines, including W95/Regswap [8] and MetaPHOR [24], explored register substitution and code reorganization. Instruction substitution, insertion, and control-flow transformations underpin the techniques summarized here [7,8,23,10,15].

Polymorphic encoding provides a related but distinct representation strategy. Analyses of SGN describe both the variation in its decoding stub and opportunities to recognize it [9,18]; memory-extraction research addresses the body recovered during execu-

tion [11]. For metamorphic programs, detection studies have considered Hidden Markov Models [27,8], API-trace similarity [29], and genetic-algorithm-based methods [14].

More recent work examines LLM-based assembly transformation, including the MetamorphASM benchmark and its MAD dataset [19,16]. Compiler-based systems such as Obfuscator-LLVM [15] offer another setting, with access to source-level or intermediate information during compilation.

The contribution of the conference paper was its payload-level *bin2bin* study, the PII design, and the joint evaluation of selected transformation configurations [4]. Those contributions are the starting point of this manuscript, rather than new claims of the extended version.

The extensions additionally connect data identification with bounded symbolic and static analysis [2], and distinguish LLM classification from assembly transformation. Assembly wrapping is attributed to its existing description and acknowledged antecedents in overlapping instructions [1]; its integration into Zone is assessed as an implementation contribution. These new directions build on the earlier payload-level study while preserving its detailed results for comparison.

## 14 Conclusions

The original Zone design and its 102-payload evaluation remain the foundation of this extended study. Their detection, working-rate, size, entropy, and similarity observations show why representation change and empirical functionality require separate measurements. The PII design and historical SGN comparison are retained within their original experimental boundaries.

The extensions add explicit code–data identification, LLM-assisted transformation, and wrapping for both architectures. Independent annotations on six curated fixtures give exact identification in 18/18 LLM and 6/6 symbolic runs. The benign transformation experiments produce accepted outputs with correct buffers in 167/168 x64 and 138/144 x86 invocations, under different frozen versions. These results are bounded by the fixture oracle and do not establish universal semantic preservation.

The broader wrapping campaign builds 243/306 variants and passes 170/306 harness checks, despite complete wrapping success on the small benign corpus. On nineteen selected real payloads, final-output identity and reported range agreement also differ. Together these observations expose the limits of extrapolation from curated tests and the need to separate classification, construction, and execution. Completed raw-file scans characterize a separate earlier cohort; the ongoing full-payload DeepSeek and current scan campaigns remain incomplete at the declared cutoff. Their final outcomes must be assessed with their own versions, budgets, and denominators.

**Acknowledgments.** This work was partially funded by the University of Perugia's project "Fighting Cybercrime with OSINT" (FICO), Civil Safety and Security for Society (WP2).

**Disclosure of Interests.** *To be completed by the authors before submission.*

## References


1. 0x4ndr3: Assembly wrapping: A technique for anti-disassembly. Author's technical article, June 8 (2020), https://0x4ndr3.github.io/posts/assembly-wrapping-a-new-technique-for-anti-disassembly/, accessed 13 November 2026
2. angr Project: Control-flow graph recovery. Official angr documentation, https://docs.angr.io/en/latest/analyses/cfg.html, accessed 13 November 2026
3. Austin, T.H., Filiol, E., Josse, S., Stamp, M.: Exploring hidden markov models for virus analysis: A semantic approach. In: 2013 46th Hawaii International Conference on System Sciences. pp. 5039–5048 (2013). https://doi.org/10.1109/HICSS.2013.217
4. Bistarelli, S., Di Benedetto, J., Santini, F.: On automating the obfuscation of malware payloads: Zone, a bin2bin metamorphic engine. In: Proceedings of the 23rd International Conference on Security and Cryptography. vol. 1, pp. 216–227. SCITEPRESS (2026). https://doi.org/10.5220/0015170600004103
5. Campion, M., Dalla Preda, M., Giacobazzi, R.: Learning metamorphic malware signatures from samples. J. Comput. Virol. Hacking Tech. **17**(3), 167–183 (2021)
6. Cho, I., Towey, D., Kar, P.: Using obfuscators to test compilers: A metamorphic experience. In: 47th IEEE Annual Computers, Software, and Applications Conference, COMPSAC 2023. pp. 1786–1791. IEEE (2023)
7. Desai, P.: Towards an Undetectable Computer Virus. Master's project, San Jose State University (2008). https://doi.org/10.31979/etd.z942-squp, https://scholarworks.sjsu.edu/etd_projects/90/
8. Desai, P., Stamp, M.: A highly metamorphic virus generator. International Journal of Multimedia Intelligence and Security **1**(4), 402–427 (2010)
9. EgeBalci: Shikata Ga Nai encoder ported into Go with several improvements (2023), https://github.com/egebalci/sgn
10. es3n1n: Obfuscating native code for fun: Part 1 - introduction (2023), https://blog.es3n1n.eu/posts/obfuscator-pt-1/
11. Farley, R., Wang, X.: CodeXt: automatic extraction of obfuscated attack code from memory dump. In: Information Security: 17th International Conference, ISC 2014, Hong Kong, China, October 12-14, 2014. Proceedings 17. pp. 502–514. Springer (2014)
12. Geng, J., Wang, J., Fang, Z., Zhou, Y., Wu, D., Ge, W.: A survey of strategy-driven evasion methods for PE malware: Transformation, concealment, and attack. Comput. Secur. **137**, 103595 (2024)
13. Glynos, D.A.: Context-keyed payload encoding: Fighting the next generation of IDS. In: Athens IT Security Conference (ATH. C0N) (2010)
14. Javaheri, D., Lalbakhsh, P., Hosseinzadeh, M.: A novel method for detecting future generations of targeted and metamorphic malware based on genetic algorithm. IEEE Access **9**, 69951–69970 (2021)
15. Junod, P., Rinaldini, J., Wehrli, J., Michielin, J.: Obfuscator-LLVM: Software protection for the masses. In: 2015 IEEE/ACM 1st International Workshop on Software Protection. pp. 3–9. IEEE (2015)
16. Madani, P.: Metamorphic malware evolution: The potential and peril of large language models. In: 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications, TPS-ISA 2023, Atlanta, GA, USA, November 1-4, 2023. pp. 74–81. IEEE (2023)
17. Madenur Sridhara, S., Stamp, M.: Metamorphic worm that carries its own morphing engine. J. Comput. Virol. **9**(2), 49–58 (2013)
18. Miller, S., Reese, E., Carr, N.: Shikata Ga Nai encoder still going strong (2019), https://cloud.google.com/blog/topics/threat-intelligence/shikata-ga-nai-encoder-still-going-strong/

19. Mohseni, S., Mohammadi, S., Tilwani, D., Saxena, Y., Ndawula, G.K., Vema, S., Raff, E., Gaur, M.: Can LLMs obfuscate code? a systematic analysis of large language models into assembly code obfuscation. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 39, pp. 24893–24901. AAAI Press (2025). https://doi.org/10.1609/aaai.v39i23.34672
20. Nawaz, M.S., Fournier-Viger, P., Nawaz, M.Z., Chen, G., Wu, Y.: MalSPM: Metamorphic malware behavior analysis and classification using sequential pattern mining. Comput. Secur. **118**, 102741 (2022)
21. Python Software Foundation: difflib: Helpers for computing deltas. Python documentation, https://docs.python.org/3/library/difflib.html, accessed 13 November 2026
22. Sathyanarayan, V.S., Kohli, P., Bruhadeshwar, B.: Signature generation and detection of malware families. In: Information Security and Privacy: 13th Australasian Conference, ACISP 2008, Wollongong, Australia, July 7-9, 2008. Proceedings 13. pp. 336–349. Springer (2008)
23. Szor, P., Ferrie, P.: Hunting for metamorphic. In: Virus bulletin conference (2001)
24. The Mental Driller: Metamorphic permutating high-obfuscating reassembler (2002), http://dsr.segfault.es/stuff/website-mirrors/29A/29a-6/29a-6.602
25. Venkatesan, A.: Code Obfuscation and Virus Detection. Master's project, San Jose State University (2008), https://scholarworks.sjsu.edu/etd_projects/116/
26. Vishwas, G., Nithin, N.S., Varshith, P., Belwal, M.: Unveiling the world of code obfuscation: A comprehensive survey. In: 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS). pp. 1–8. IEEE (2023)
27. Wong, W., Stamp, M.: Hunting for metamorphic engines. Journal in Computer Virology **2**, 211–229 (2006)
28. Zeng, D., Tan, G.: From debugging-information based binary-level type inference to CFG generation. In: Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy. pp. 366–376. CODASPY '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3176258.3176309
29. Zhang, Q., Reeves, D.S.: MetaAware: Identifying metamorphic malware. In: Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007). pp. 411–420. IEEE (2007)